Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

3. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.

- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction
Construct an MLP with
- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training
Compile using
- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation
Compute
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

The complete source code is attached separately in the GitHub repository and submitted along with this report.

GitHub Repository:
`https://github.com/Sadhana2006-art/Deep-Learning-Lab`

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.1
Batch Size	32
Optimizer	sgd
Activation Function	Tanh
Epochs	30
Dropout	0.0
Cross-validation Accuracy	0.8863 (88.63%)
Testing Accuracy	0.8862 (88.62%)

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.888100	0.886200
Precision	0.888441	0.888336
Recall	0.888100	0.886200
F1-score	0.888136	0.885920
Training Time	151.424753	162.067963

8. Plots with Inference

0.1 Sample Images

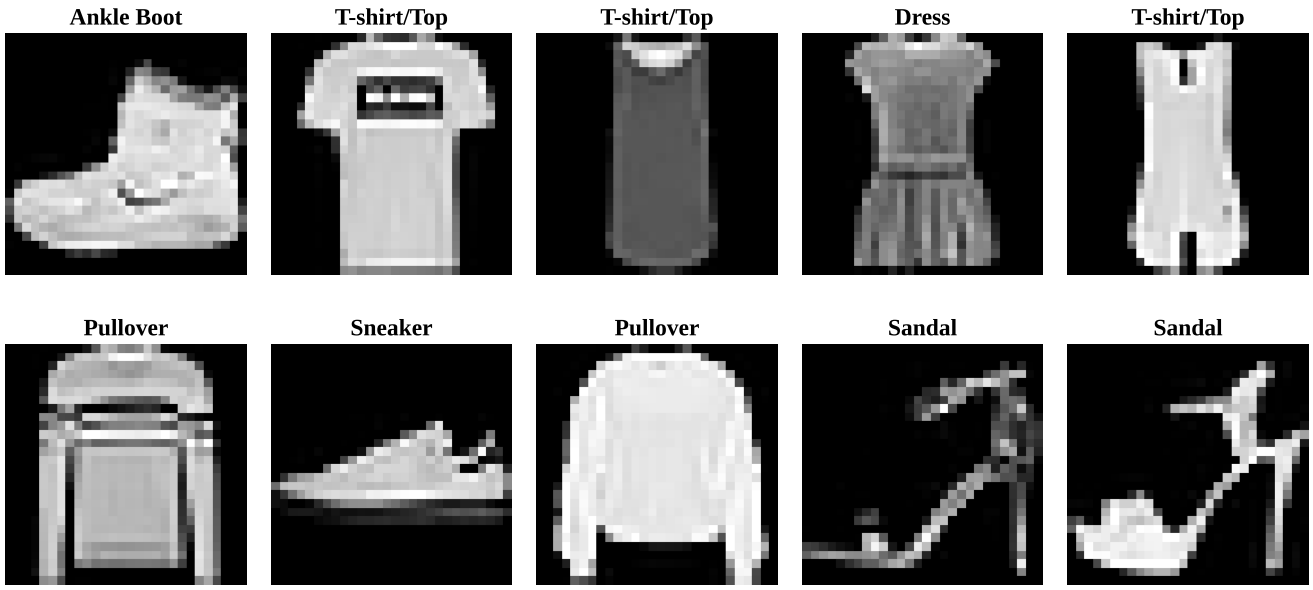

Figure 1: Sample Images from the Dataset

Interpretation:

Figure displays ten sample images from the Fashion-MNIST dataset, which are different types of clothing such as t-shirts, dresses, pullovers, sandals, sneakers and ankle boots. The images are in grey-scale with a resolution of 28×28 pixels.This confirms that the dataset is loaded properly.

0.2 Class Distribution

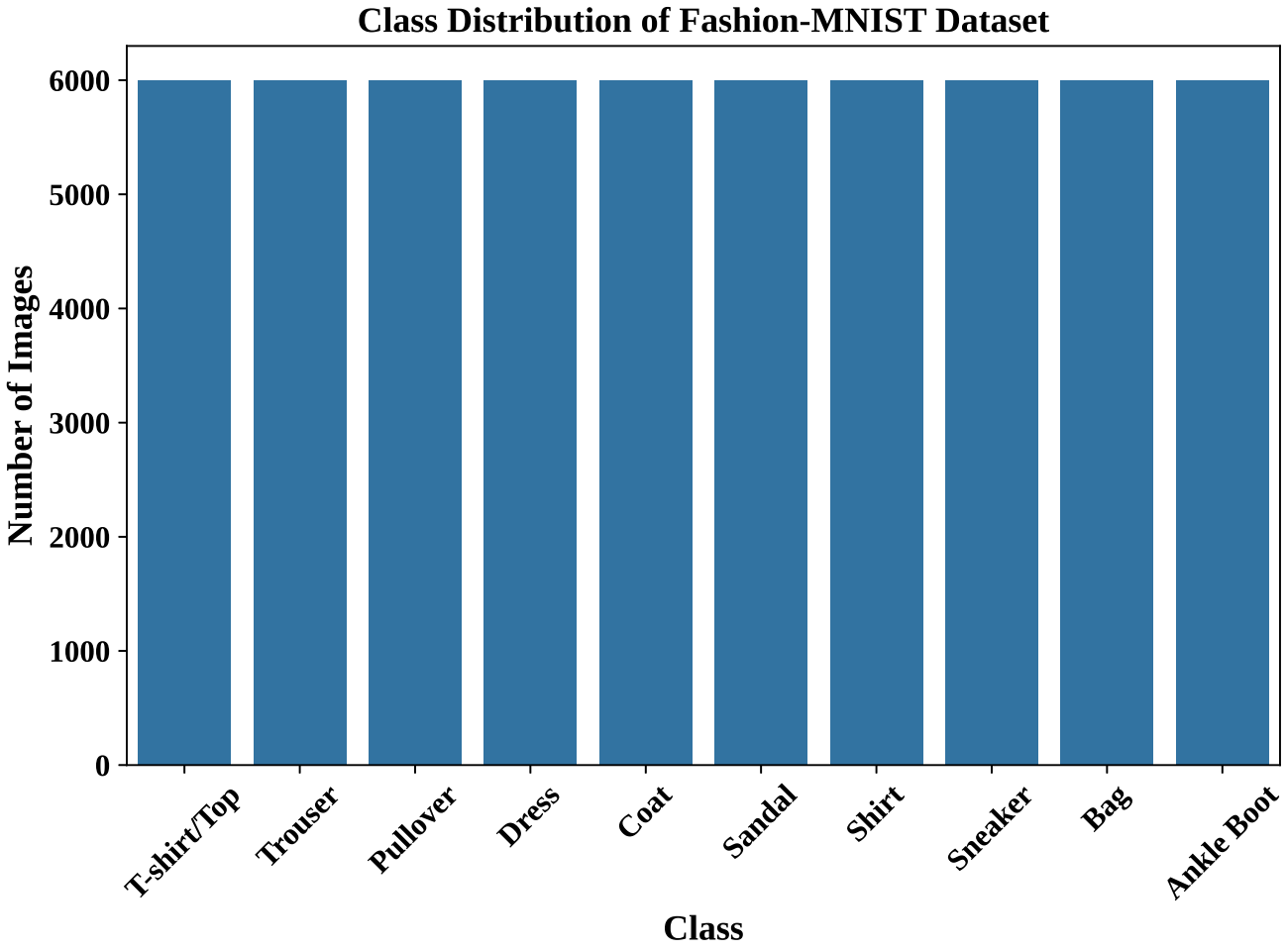

Figure 2: Distribution of Classes in the Dataset

Interpretation:

The dataset is evenly distributed which is a balanced dataset among the ten classes, with each class having around 6,000 images. This uniform distribution helps ensure fair model training and improves the robustness of the classification results.

0.3 Training Accuracy vs Epoch

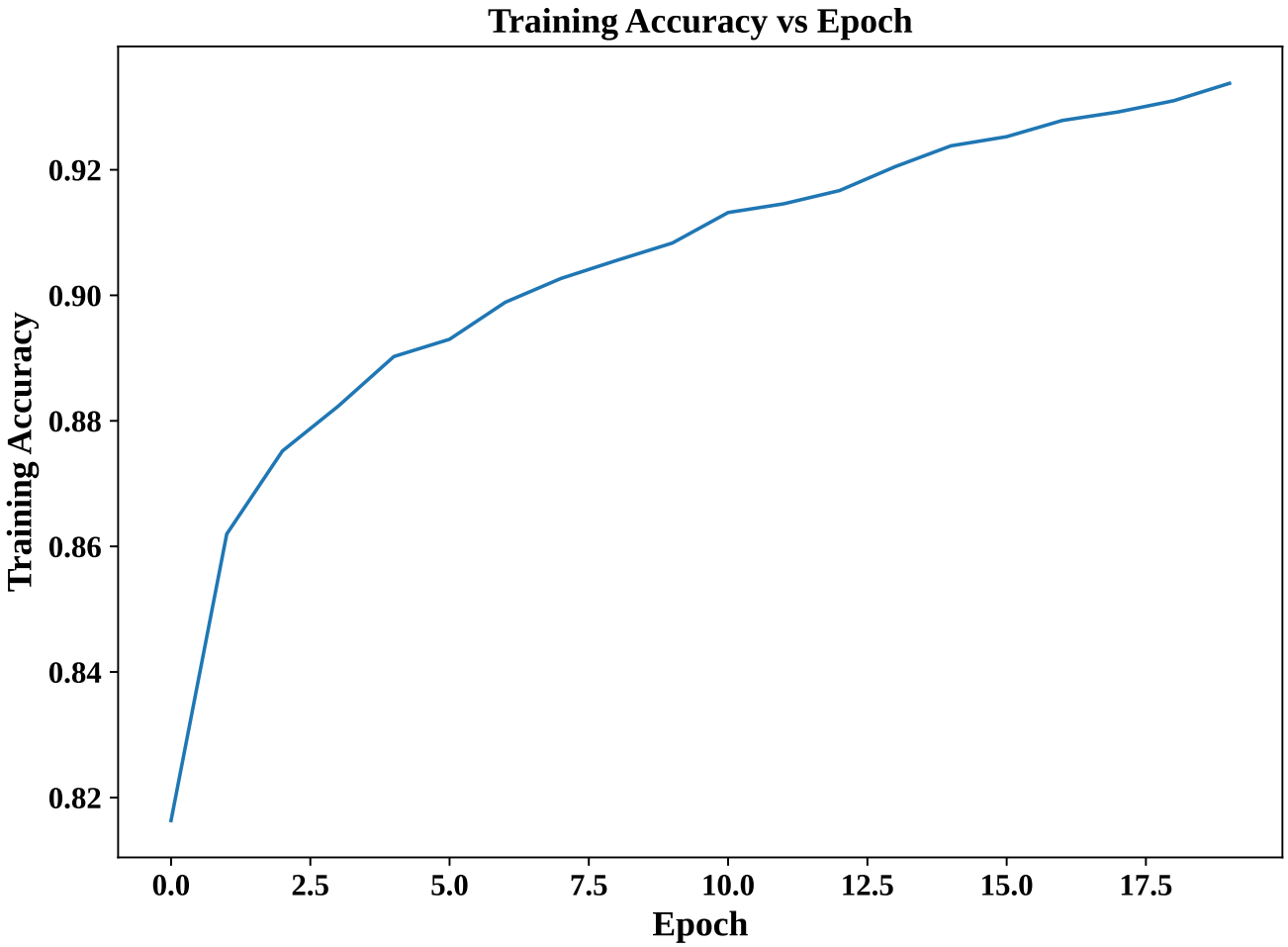

Figure 3: Training Accuracy versus Epoch

Interpretation:

The training accuracy shows a gradual growth through the 20 epochs indicating that the MLP model is learning well. The curve gradually stabilises in the later epochs, suggesting that the model is converging to an optimal solution without significant fluctuations.

0.4 Validation Accuracy vs Epoch

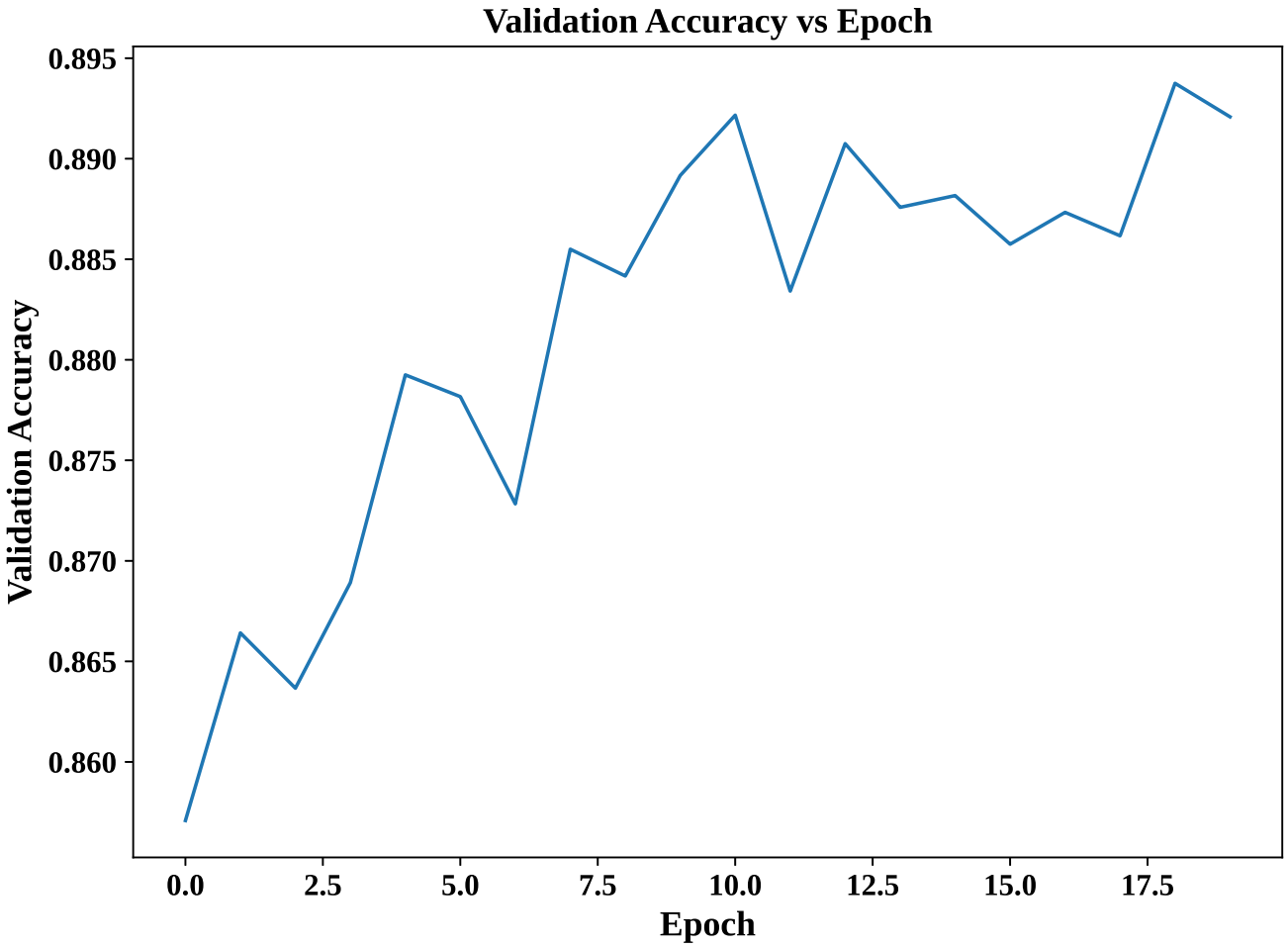

Figure 4: Validation Accuracy vs Epoch

Interpretation:

The validation accuracy increases gradually during training and converges to an approximately 89% accuracy, which shows that the model learns well and generalises well.

0.5 Training Loss vs Epoch

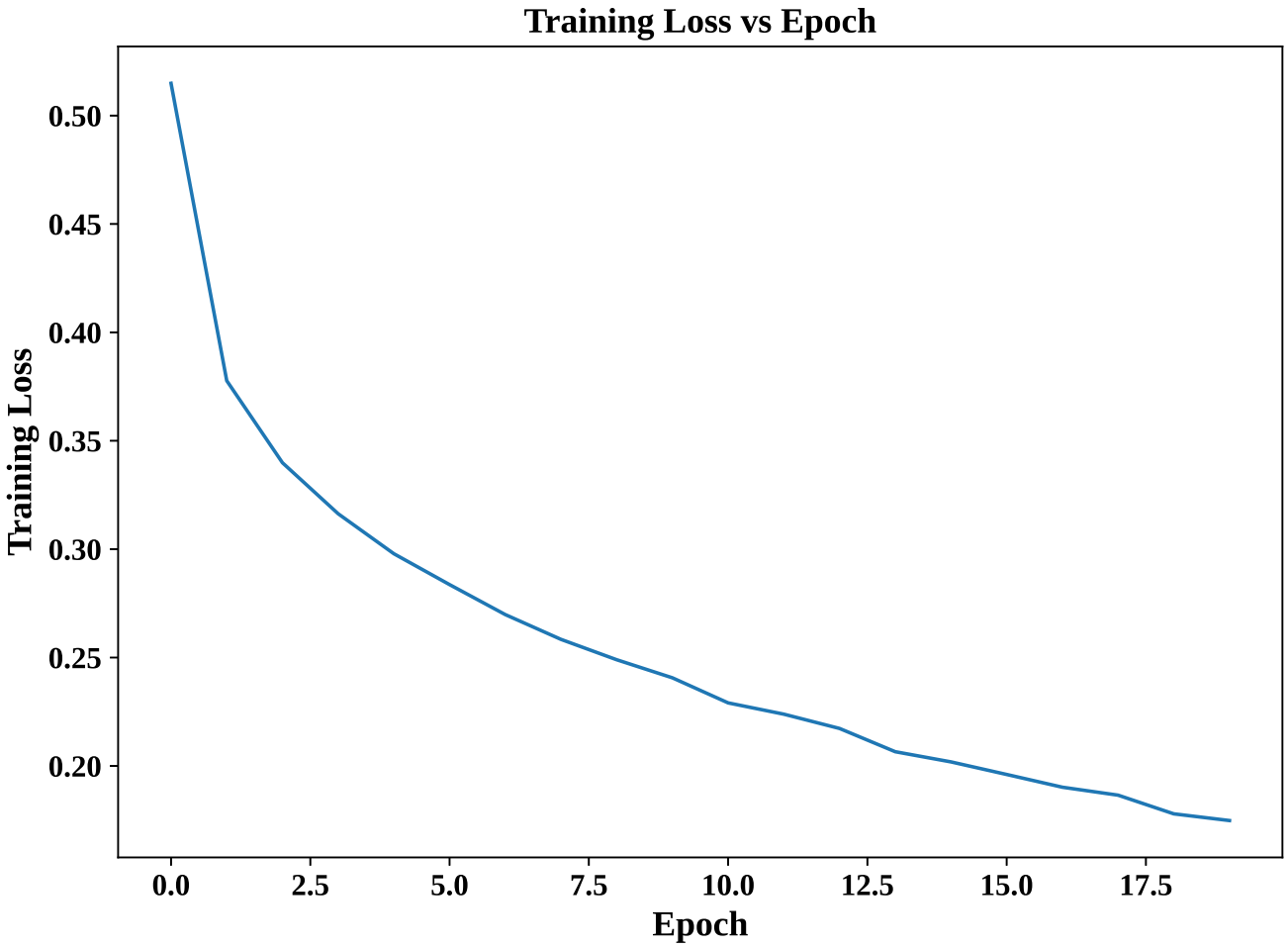

Figure 5: Training Loss versus Epoch

Interpretation:

The training loss decreases consistently during the training, which means that the MLP is improving its predictions by minimising the error. The absence of sudden spikes indicates that the optimisation is stable and the learning algorithm converges successfully.

0.6 Validation Loss vs Epoch

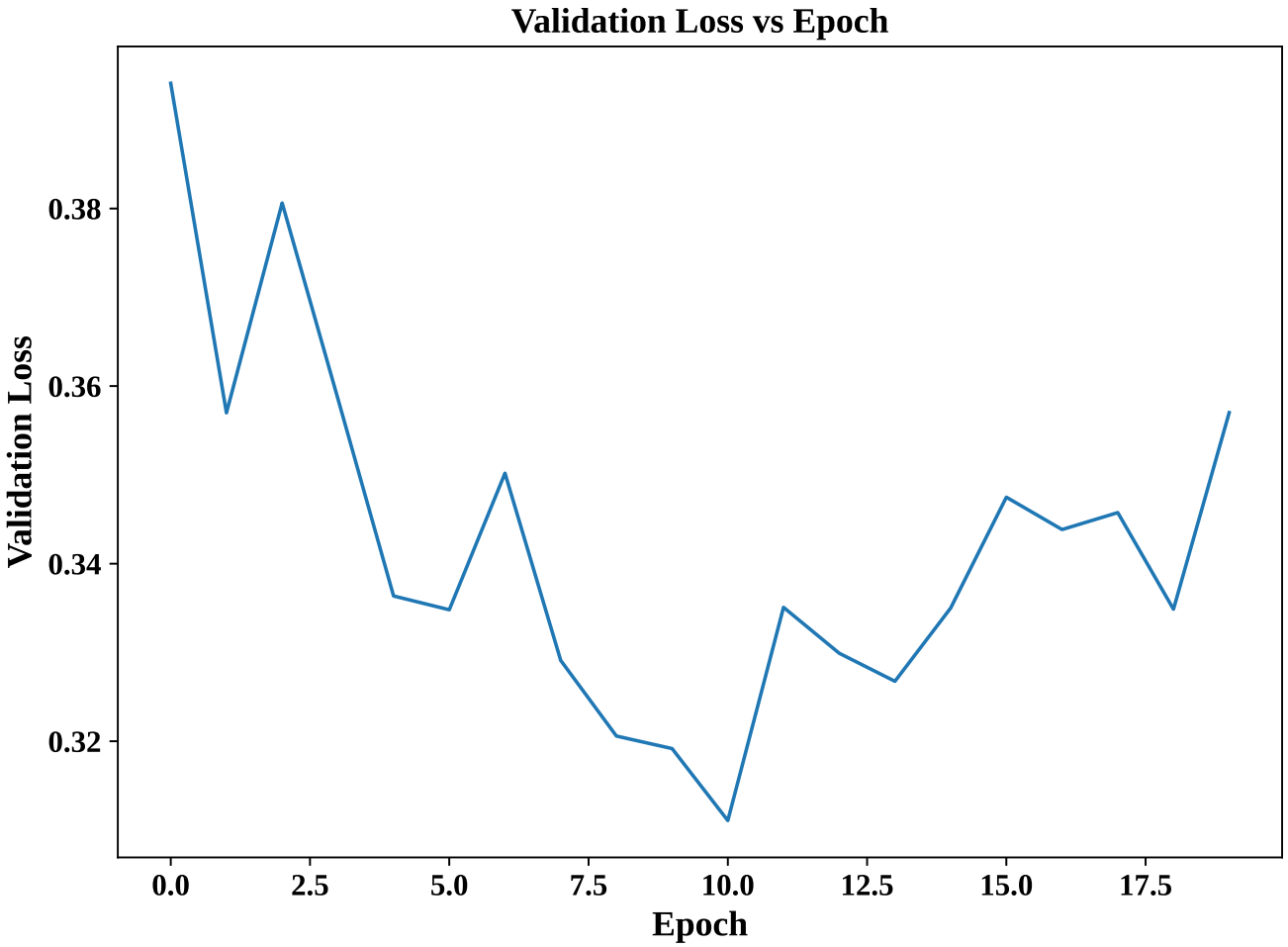

Figure 6: Validation Loss versus Epoch

Interpretation:

In the first few training epochs, the validation loss decreases and reaches its minimum around the middle of training, which indicates an improvement in performance on unseen validation data. The minor fluctuations in the last epochs indicate marginal changes in generalisation and the model is stable without significant overfitting.

0.7 Confusion Matrix

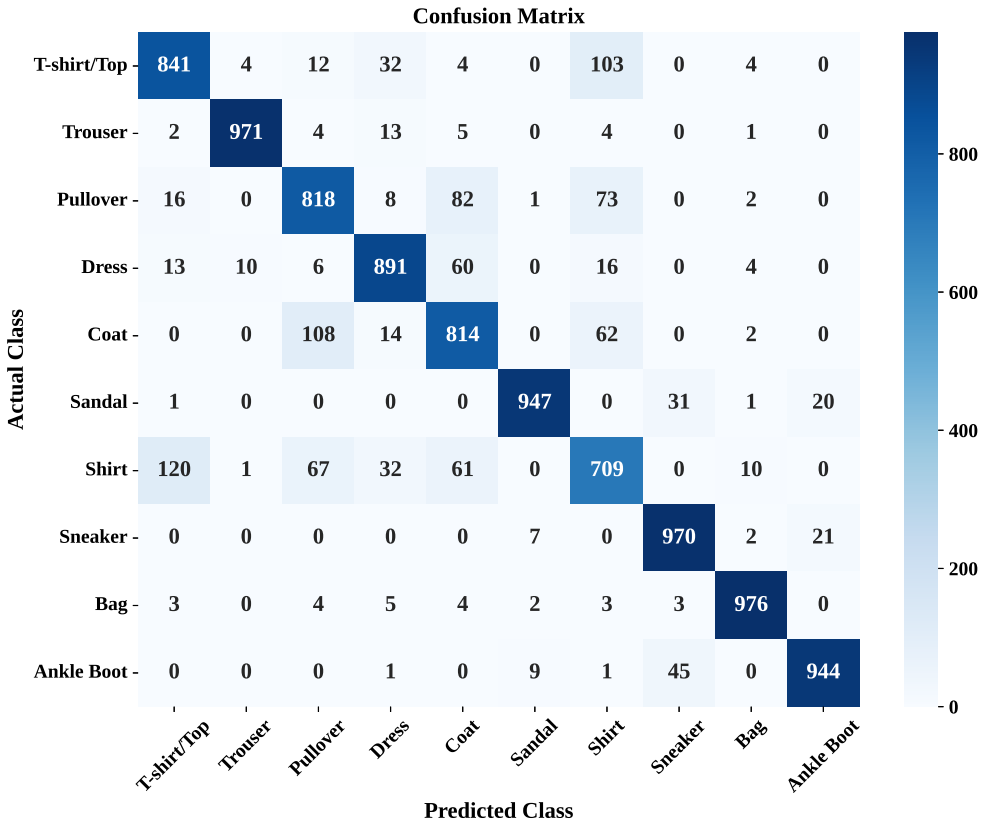

Figure 7: Confusion Matrix

Interpretation:

It is seen from the confusion matrix that the predictions tend to lie on the diagonal, which means that the MLP correctly predicts most of the images. Some mistakes in classification happen between those classes that look alike, namely T-shirt/Top – Shirt, Pullover – Coat, and Sneaker – Ankle Boot.

0.8 Hyperparameter Search Results

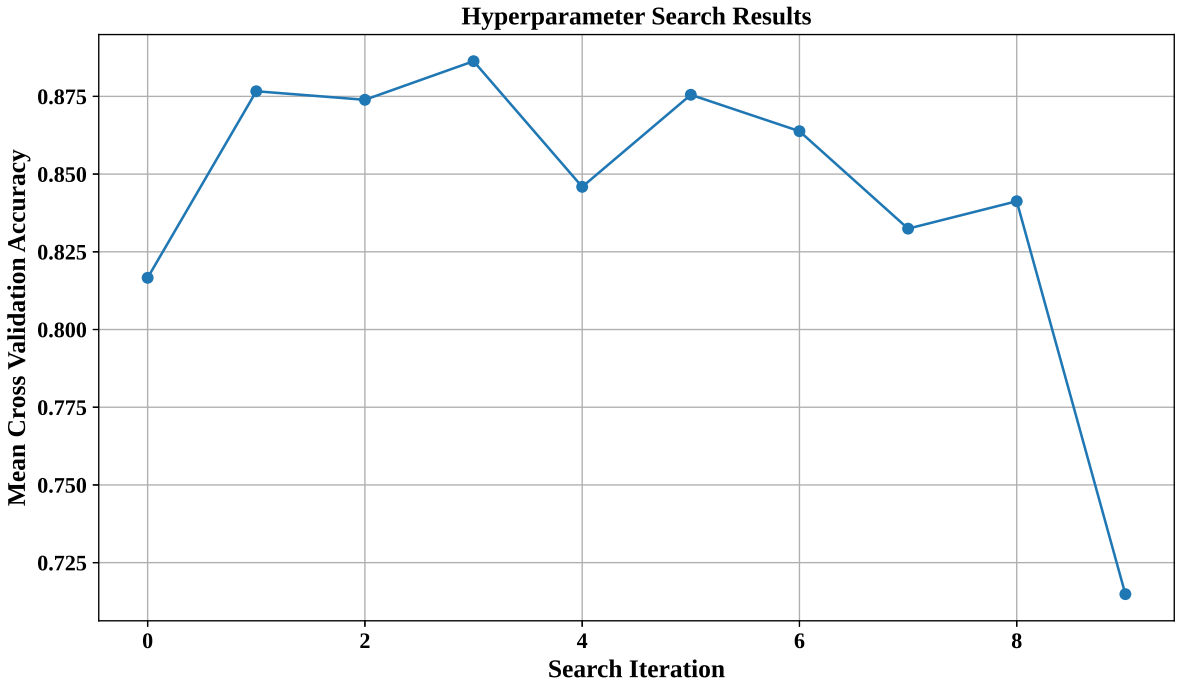

Figure 8: Hyperparameter Search Results

Interpretation:

Multiple combinations of hyperparameters were tried during the Randomized Search and this was done using 5-fold cross validation. Out of all the different combinations tried, Iteration 3 produced the best mean cross validation accuracy of 88.63% and this combination was used to train the optimized MLP.

0.9 Best Model Accuracy Comparison

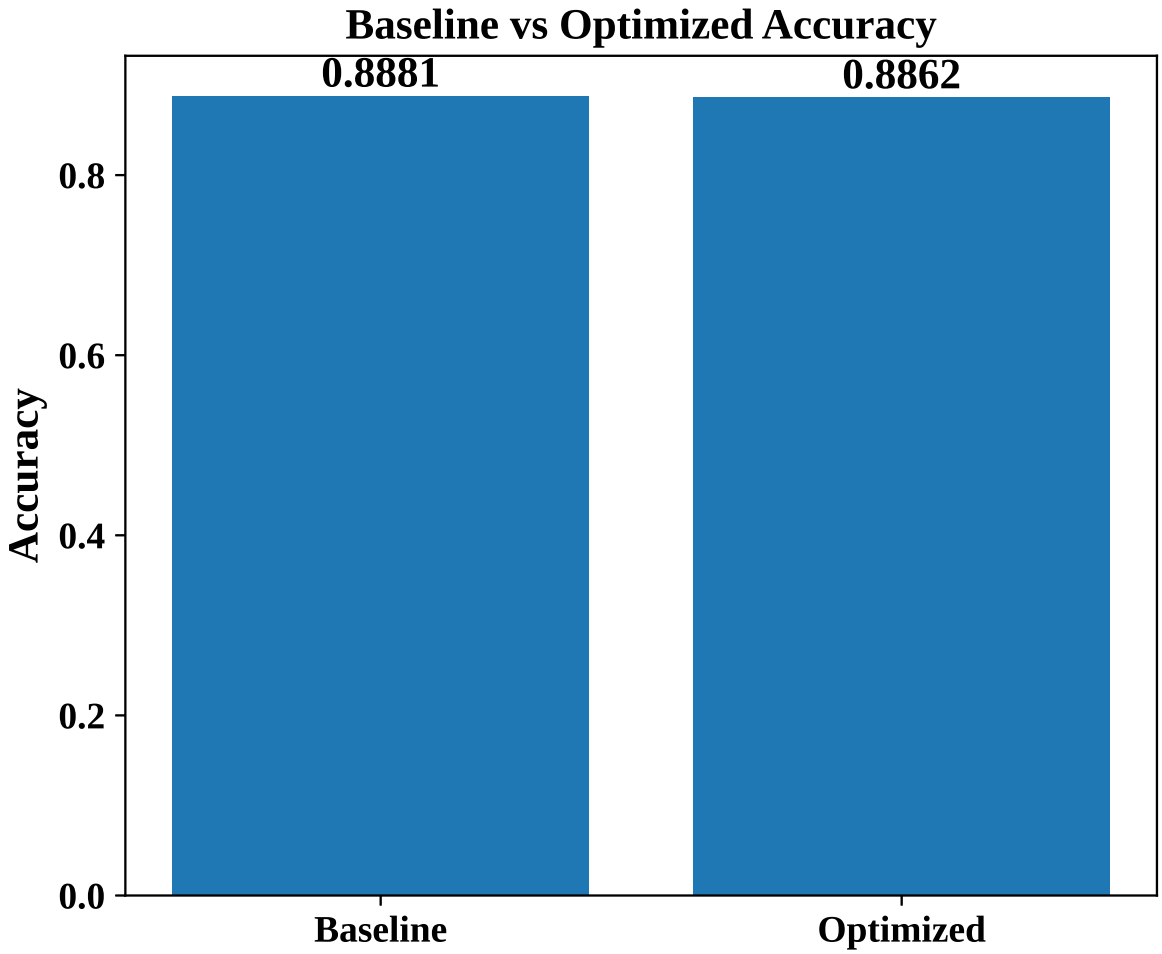

Figure 9: Best Model Accuracy Comparison

Interpretation:

The accuracy for the optimized MLP model on the test set was 88.62% compared to 88.81% of the baseline model, meaning that there was no significant change in performance but only in the model's architecture through hyperparameter optimization.

9. Discussion

1. **Which optimization method was used?**
   RandomizedSearchCV with 5-fold cross-validation along with SciKeras KerasClassifier wrapper was utilized. In this method, a random combination of hyperparameters is picked and the model that gives maximum accuracy in terms of cross-validation is chosen.
2. **Which hyperparameters were selected?**
   In Randomized Search, one hidden layer, 128 neurons, Tanh activation, SGD optimization with learning rate of 0.1, batch size of 32, number of epochs of 30, and dropout of 0.0 were chosen which resulted in a cross-validation accuracy of 88.63%.
3. **Did optimization improve performance?**

In comparison to the baseline model whose accuracy is 88.81% in testing, the optimized model's accuracy is only 88.62%. Thus, optimization does not result in a better final testing accuracy but a comparable model is obtained with a hyperparameter setting that is selected automatically.

4. **Which hyperparameter had the greatest impact?**
   The optimizer, learning rate, and activation function made the most significant effect on the results of the neural network. The combination of various values of these hyperparameters led to differences in the results of cross-validation.
5. **Compare Grid Search and Randomized Search.**
   Grid Search tries all possible values of hyperparameters, which makes it costly computationally but exhaustive. On the other hand, Randomized Search considers only randomly selected values of hyperparameters, which makes it less costly and faster but still generates near-optimal results.
6. **Recommend the best MLP configuration.**
   According to the Randomized Search, the suggested MLP architecture is as follows:
   Input Layer: 784 nodes, Hidden Layers: 1, Hidden Nodes: 128, Activation Function: Tanh, Optimizer: SGD, Learning rate: 0.1, Batch size: 32, Epochs: 30, Dropout: 0.0, Output Layer: 10 nodes with Softmax activation
   It provided the highest cross-validation accuracy (88.63%) while keeping the testing accuracy (88.62%) equal to that of the baseline architecture.

10. Conclusion

A Multi-Layer Perceptron (MLP) model has been successfully built for multi-class classification of images based on the Fashion-MNIST data set. The images in the data set were preprocessed in terms of flattening, normalization, and one-hot encoding of the labels. The baseline model obtained good classification results and RandomizedSearchCV with the help of the SciKeras package was applied for automatic tuning of hyperparameters. The tuned model obtained 88.63% cross-validation accuracy which is comparable to the baseline model's test accuracy.

11. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.